



Technical University of Denmark

DTU Compute

Department of Applied Mathematics and Computer Science

Indexing & Hashing

Performance Optimization of Relational File Systems

Parts of Chapters 4.6, 14 and 24 in 7th edition of textbook

Parts of Chapters 4.5.3 and 11 in 6th edition of text book

©Anne Haxthausen

These slides have been prepared by Anne Haxthausen, partly reusing/modifying some slides by Flemming Schmidt.
Some examples come from Silberschatz, Korth, Sudarshan, 2010.

Contents

- Background Knowledge: File Structure and Organization
(from book chapter 13 in 7th edition / chapter 10 of 6th edition, but no need for you to read)
- Indexing
- Hashing

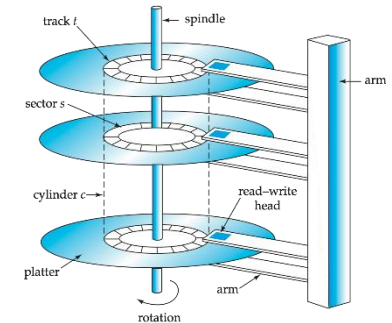
File Structure, Organization and Storage

■ File Structure

- A *file* is a sequence of records.
- A *record* is a sequence of *fields*.

■ Storage of a database

- A *database* is stored in *files* on a *disk*:
 - Each *table (relation)* is stored in a *file*:
 - Each *row* is stored in a *record* of the file:
 - Each *attribute value* is stored in a *field* in the record.
- The files are partitioned into fixed-length storage units called *blocks*.



Example:	Block 1	record 0	10101	Srinivasan	Comp. Sci.	65000
		record 1	12121	Wu	Finance	90000
		record 2	15151	Mozart	Music	40000
	Block 2	record 3	22222	Einstein	Physics	95000
		record 4	32343	El Said	History	60000
		record 5	33456	Gold	Physics	87000
	Block 3	record 6	45565	Katz	Comp. Sci.	75000

File Organization

- **File organization** refers to the way records are organized in a file.

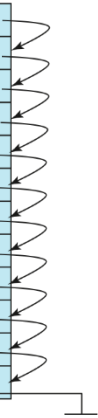
- **Heap File**

- A record can be placed anywhere in the file where there is space.

- **Sequential File**

- Records are stored *in sequential order*, based on the value of the *search key* of each record.
 - Example - Instructor table with InstID as search key: InstID

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



- **Hash File**

- A hash function computed on some attribute of each record; The result specifies in which block of the file the record should be placed.

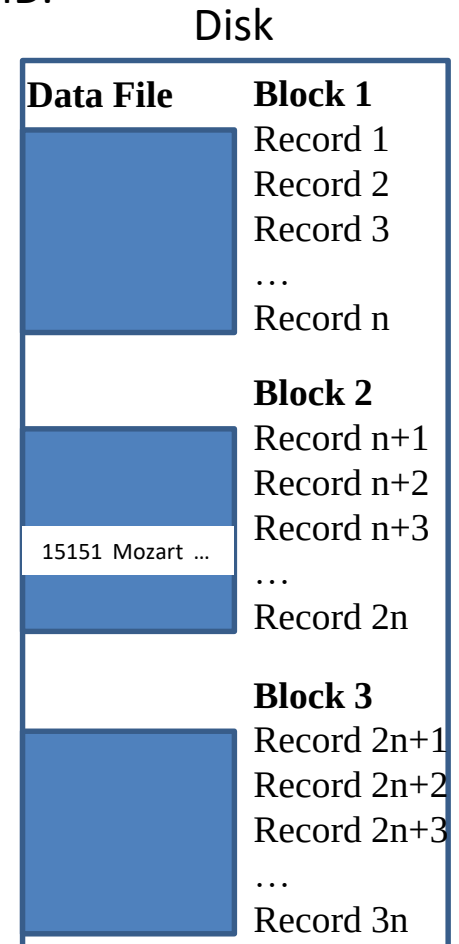
How to Find a Record Fast?

- SELECT statements give rise to searching for Records.

Example: **SELECT** InstName **FROM** Instructor **WHERE** InstID = 15151;

- Find the Record with *Search Key* 15151 for Instructor.InstID.
- The Instructor file is in 3 Blocks on the Disk.
- One way:
Read the file sequentially, loading blocks one by one into Memory until the record is found.

- Can this be done faster?



How to Find a Record Fast for a Given Search Key?

1. Use an Index File of Search Keys

1st field: Search Key
2nd field: Pointer

The Pointer points to the start of the Block where the Record is stored, or directly to the record.

Index File

<u>Search Key</u>	Pointer
Value 1	Block 1
Value 2	Block 1
Value 3	Block 3
...	
15151	Block 2
...	

Data File

Block 1
Record 1
Record 2
Record 3
...
Record n

Block 2
Record n+1
Record n+2
Record n+3
...
Record 2n

Block 3
Record 2n+1
Record 2n+2
Record 2n+3
...
Record 3n

Index file organization: sequential (ordered), B+ tree, hash, ...

2. Use a Hashing Algorithm on the Search Key

Search Key = 15151

Hash Algorithm: $\text{Search Key} \% 3 + 1$

Block Address: $15151 \% 3 + 1 = 2$

- The $x \% y$ modulus operation is the remainder of x/y

Using hashing saves to have the index file.

Summary

- A File can be read sequentially, one block and one record after another!
 - But indexing mechanisms can be used to **speed up** access dramatically.
- A Search Key can be defined to look up records in a file
 - Search Keys can be stored in an Index File with pointers.
 - First attribute is the Search Key, and the second attribute is a Pointer.
- Index Files are much smaller than the original file
 - An Index File is loaded from disk and often fits into memory.
- Alternatively a Hashing Algorithm can be defined
 - Based on a Search Key it calculates in which Block a record has to be read from or written to.
- Index Evaluation Metrics
 - Access time, insertion time, deletion time and space overhead.

Indexing

- Concepts for Ordered Index Files
 - Primary versus Secondary Indexes
 - Dense versus Sparse Indexes
 - Multi-column Indexes
 - Multilevel Indexes
- Index File Organization:
 - Sequential Files
 - B⁺-Tree
 - ...

Index Definition in SQL

- Create an index

CREATE [UNIQUE] INDEX *<index-name>* **ON** *<table-name>*(*<attribute-list>*)

Like: **CREATE INDEX** NameIndex **ON** Instructor(InstName);

Most database systems allow a specification of **the type** of index.

Like: **CREATE INDEX** NameIndex **ON** Instructor(InstName) **USING BTREE**;

CREATE INDEX NameIndex **ON** Instructor(InstName) **USING HASH**;

Each storage engine supports some or all index types. The default storage engine INNODB only allows BTREE.

MariaDB uses BTREE as *default*, if no type is specified.

- ✓ MySQL and MariaDB will *automatically* for each table define a Primary Index on the Primary Key and Secondary Indexes for all Foreign Keys.

- To drop an index

DROP INDEX *<index-name>* **ON** *<table-name>*;

Like: **DROP INDEX** NameIndex **ON** Instructor;

- To show the index set for a table

SHOW INDEX FROM *<table-name>*:

Like: **SHOW INDEX FROM** Instructor;

Ordered Index Files

■ (Sequentially) Ordered Index

- Index File is sorted on the Search Key which is one or several attributes from the data file.
- Since it is sorted we can use binary searching to find an index.

Index File

Search Key	Pointer
1	
2	
3	

Data File (for a relation)

A1	A2	A3	A4

■ Primary Index

- Index File and Data File are both sorted on the same Search Key.
- It is the Primary Index that determines the order of the records in the Data File. The Index is also called a **Clustering Index**, since it determines the clustering of records in blocks.
- A relation has at most one Primary Index. It is usually (but not necessarily) defined on the primary key attribute. In many DBMS the primary key automatically gets an index.
- **Note:** some literature/books use the terms **primary index** and **clustering index** with different meanings!

■ Secondary Index = Index which is not Primary

- Index File and the Data File are sorted on different Search Keys.
- A Secondary Index does not determine any order on the records in the Data File.
- Exists only, if there is a Primary Index.
- A relation can have several Secondary Indexes.

Dense Versus Sparse Index Files

- Dense Index
 - The Index File has a record for every Search Key value in the Data File
- Sparse Index = Non-Dense Index
 - The Index File has **not** a record for every Search Key value in the Data File

Examples of Ordered Indexes

- Examples of ordered indexes on following slides

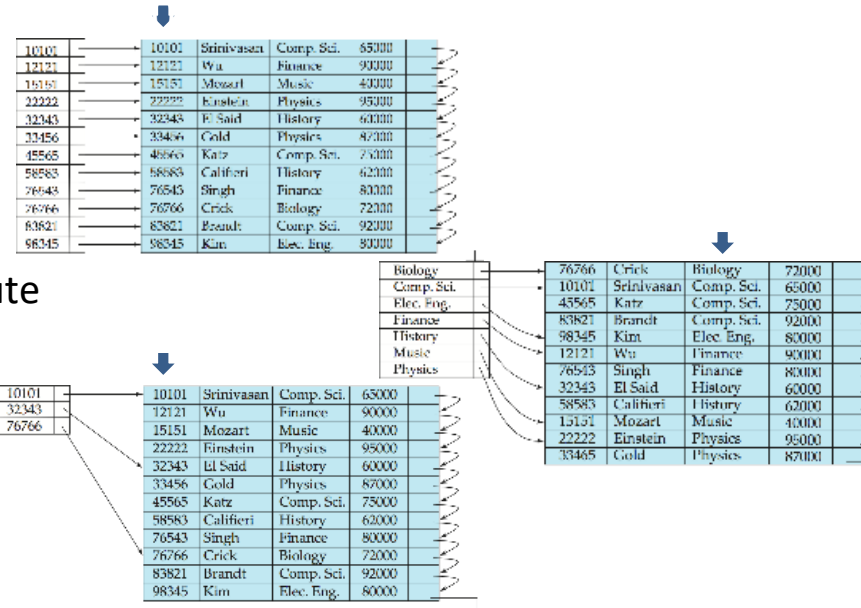
1. Primary indexes:

1. Dense index on (primary) key attribute

2. Dense index on non primary key attribute

3. Sparse index on (primary) key attribute

4. Sparse index on non key attribute

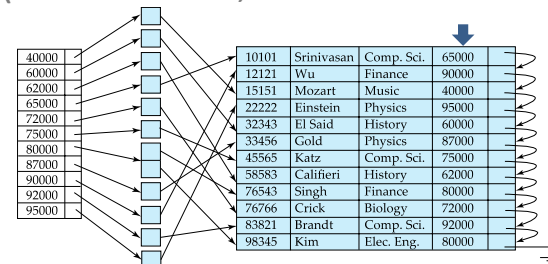


2. Secondary indexes:

1. Dense index on key attribute: no example (similar to 1.1, but with arrows crossing)

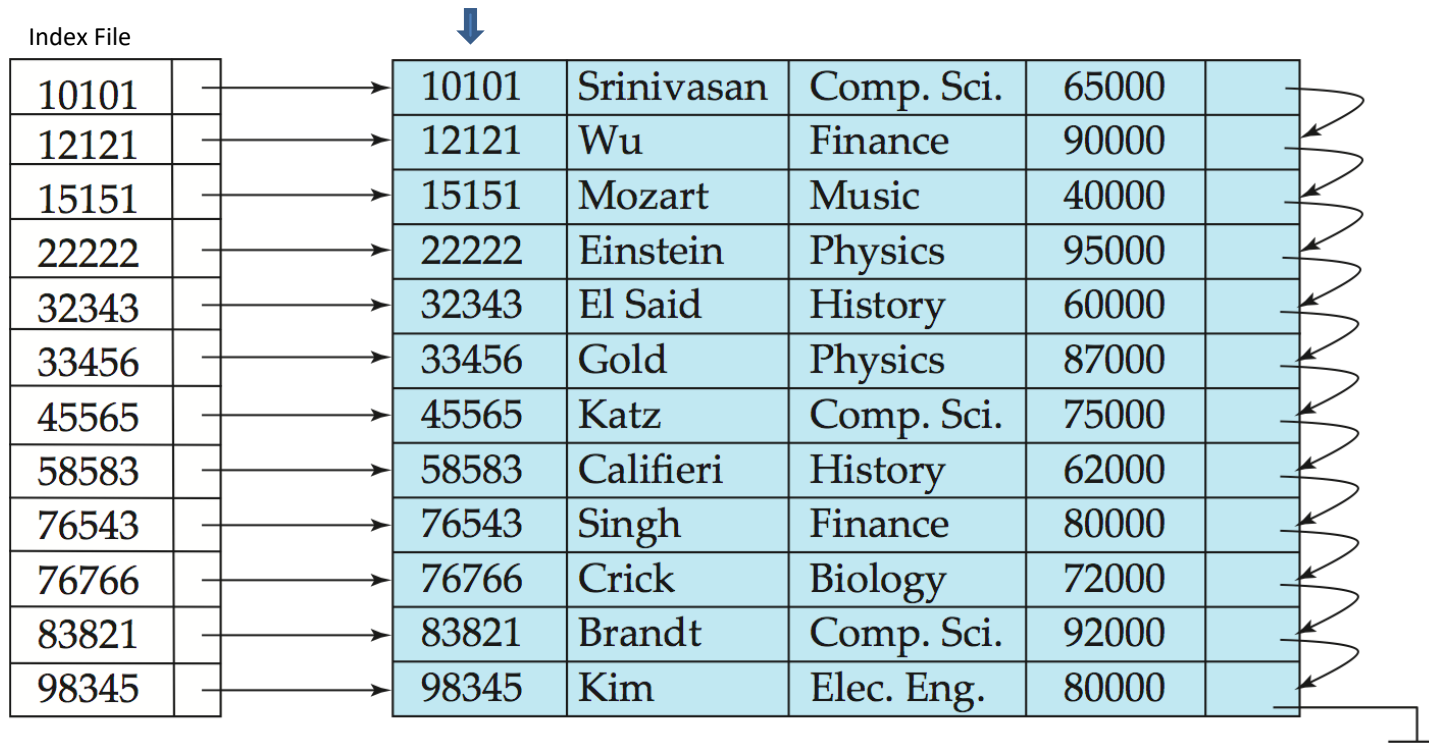
2. Dense index on non primary key attribute

3. Sparse index is **not** an option



Primary Index which is Dense

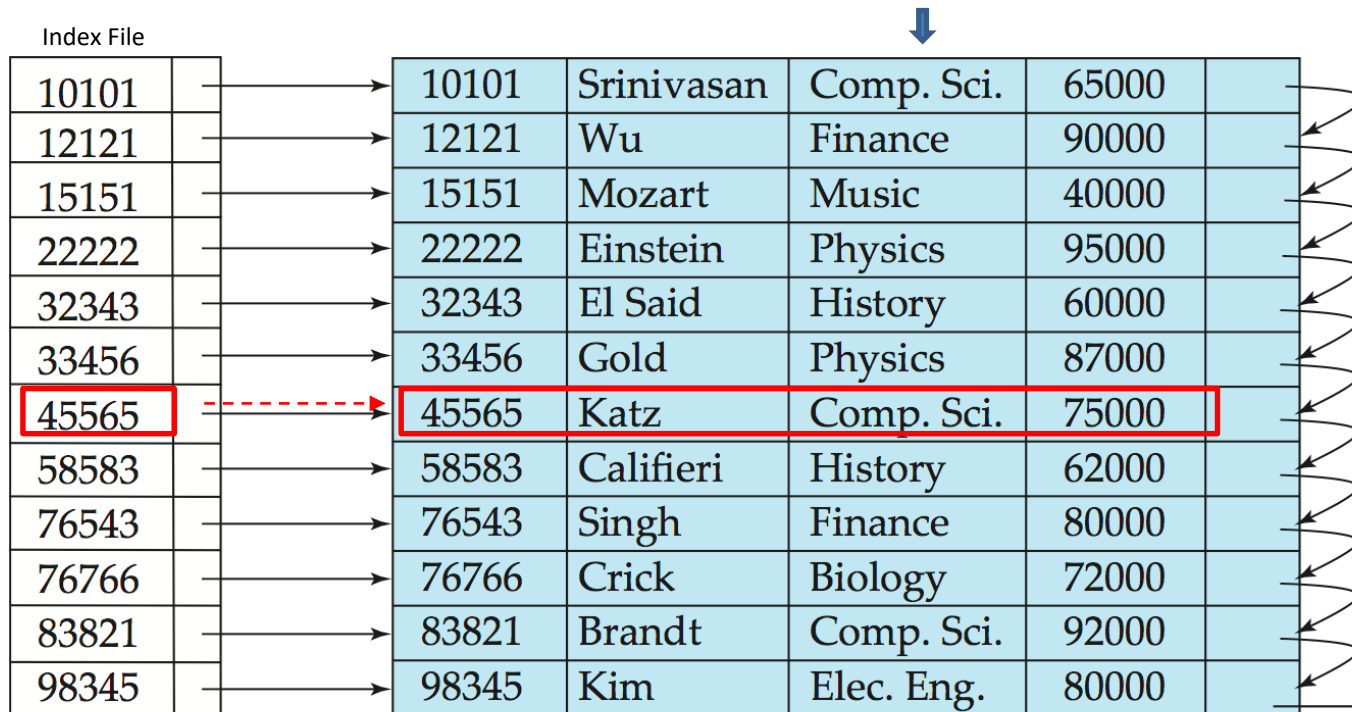
- Dense, Ordered Index on the **Primary Key** attribute Instructor.InstID



- The Index is Ordered (Index File is sorted on the Search Key).
- The Index is Primary (Data File are sorted on the same Search Key).
- The Index is Dense (All File Search Keys have an Index File Record).

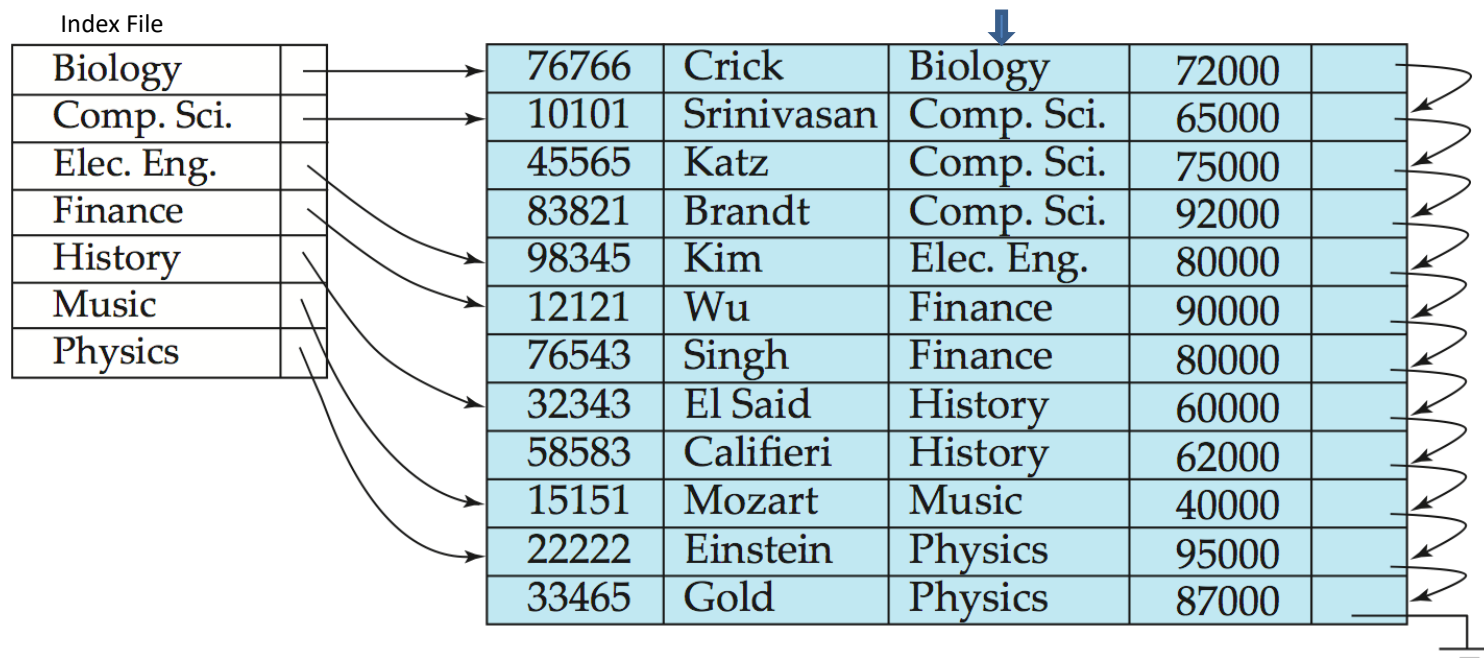
Queries Using Dense Index

- To locate records with Search Key value = K (e.g. Instructor.InstID = 45565)
 1. Find the record with Search Key value V = K in the Index File.
In practice: read the index into memory and use binary searching.
 2. Follow the associated pointer to the record in the Data File and read the record (block containing it) into main memory.



Primary Index Which Is Dense

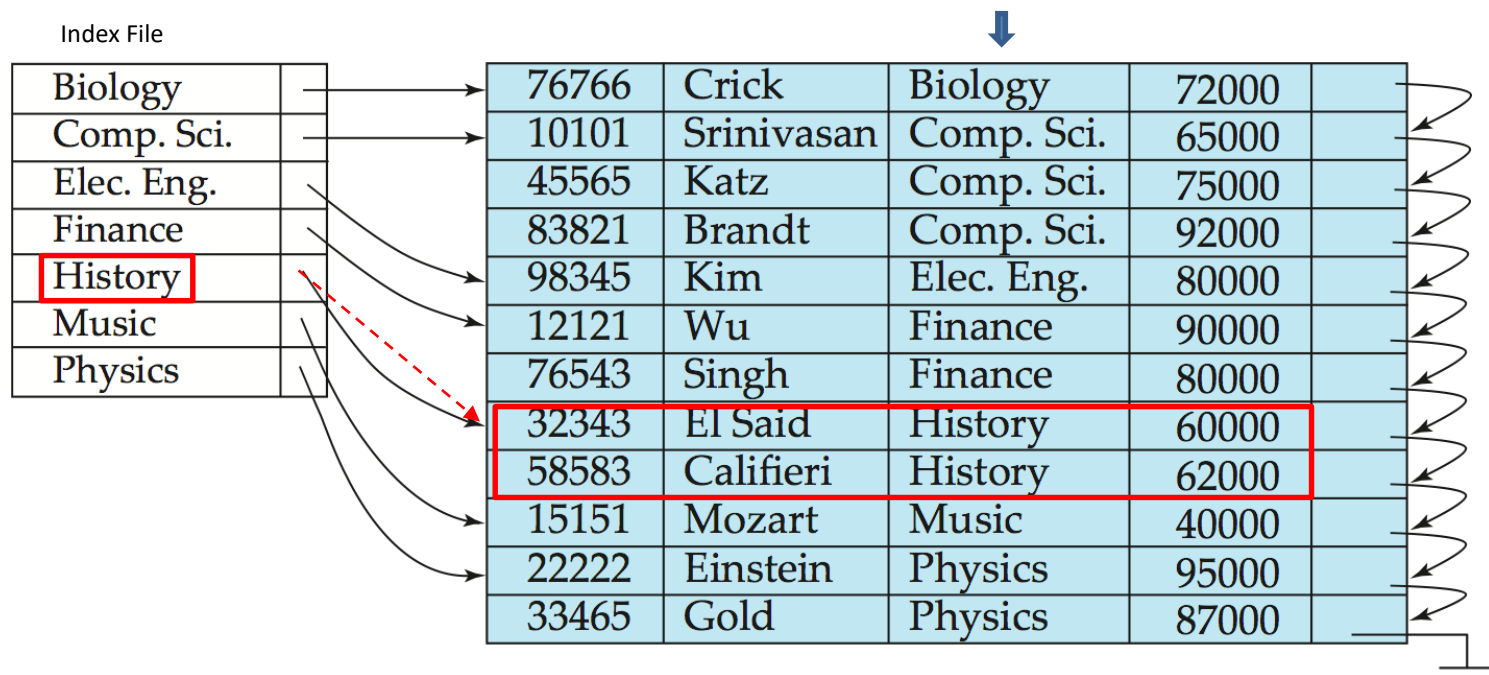
- Dense, Ordered Index on a **Non-Key** Attribute Instructor.DeptName



- The Index is Ordered (Index File is sorted on the Search Key).
- The Index is Primary (Index File and File are sorted on the same Search Key).
- The Index is Dense (Index File has a record for every Search Key value in the Data File).

Queries Using a Dense Index

- To locate records with Search Key value = K (e.g. Instructor.DeptID = History)
 1. Find the record with Search Key value V = K in the Index File.
In practice: read the index into memory and use binary searching.
 2. Follow the associated pointer to the Data File and search sequentially from there.
In practice: read the pointed block into main memory and perform searching on that. Read more blocks, if needed.



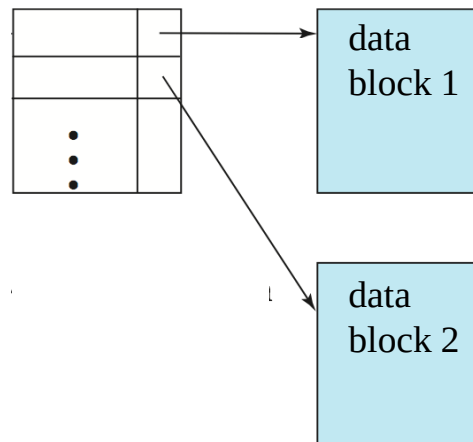
Sparse Index Files

■ Sparse Index

- By definition: The Index File contains only some of the Search Key values of the Data File.
- Only possible for Primary Indexes (Index File and Data File sorted on the same Search Key), as the same ordering is needed to locate records.

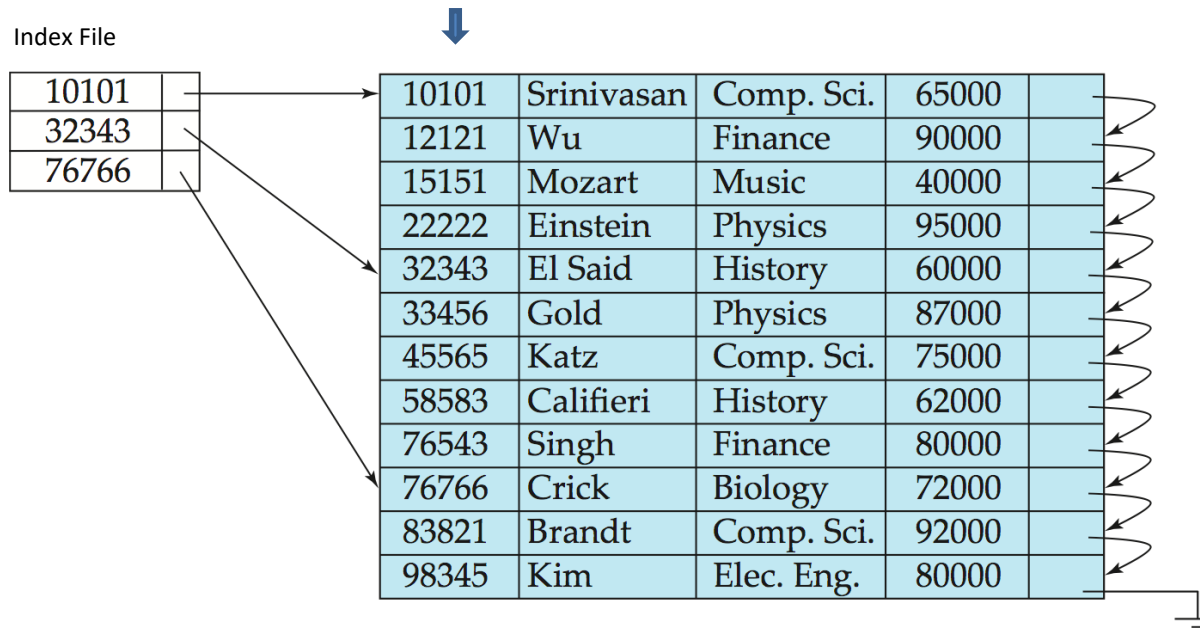
■ Which Search Keys to include in the index:

- A good choice is to include each key of the 1st record in every block in the file:
 - 1st record in index points to the 1st record in the 1st block of the data file
 - 2nd record in index points to the 1st record in the 2nd block of the data file
 - ...



Primary Index Which Is Sparse

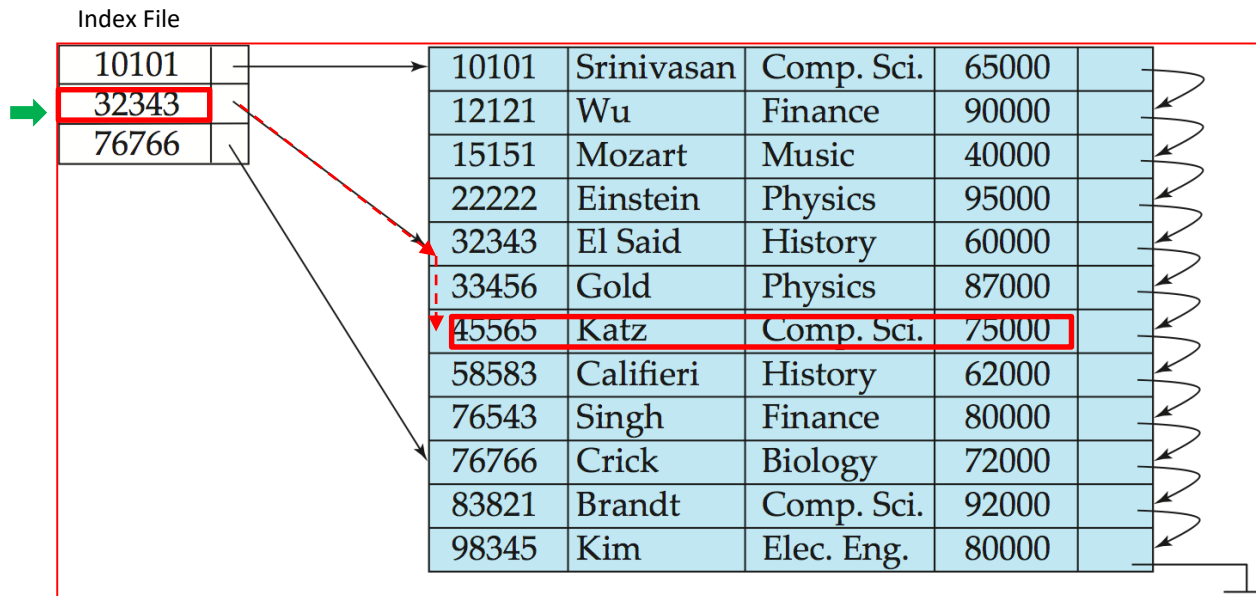
- Sparse, Ordered Index on the **Primary Key** attribute Instructor.InstID



- The Index is Ordered (Index File is sorted on the Search Key).
- The Index is Primary (Data File are sorted on the same Search Key).
- The Index is Sparse (Not all File Search Keys have an Index File Record).

Queries Using a Sparse Index

- To locate a record with Search Key value K (e.g. Instructor.InstID = 45565)
 1. Find the record with largest Search Key value $V \leq K$ ($V = 32343$) in the Index File. In practice: read the index into memory and use binary searching.
 2. Follow the pointer to a record in the data File and start sequential searching. In practice: read the pointed block into main memory and perform searching on that. If it is not in that block, read next block and search in that. And so on.



Index File Update After an Insertion in the Data File

- Single-level Index File record insertion
 - Search for the Search Key value of the record to be inserted
 - Dense Index:
If the Search Key value does not appear in the index, then insert it
 - Sparse Index:
 - If the Index File stores an entry for each block of the Data File, no change needs to be made to the Index File unless a new block is created.
 - If a new block is created, the first search key value appearing in the new block is inserted into the Index File.
- Multilevel insertion and deletion
 - Algorithms are simple extensions of the single-level algorithms

Index File Update After a Deletion in the Data File

Single-level Index

- Dense Index, example:

Biology	76766	Crick	Biology	72000
Comp. Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

- Sparse Index, example:

10101	10101	Srinivasan	Comp. Sci.	65000
32343	12121	Wu	Finance	90000
76766	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	32343	El Said	History	60000
	33456	Gold	Physics	87000
	45565	Katz	Comp. Sci.	75000
	58583	Califieri	History	62000
	76543	Singh	Finance	80000
	76766	Crick	Biology	72000
	83821	Brandt	Comp. Sci.	92000
	98345	Kim	Elec. Eng.	80000

Sparse Index Compared to Dense Index

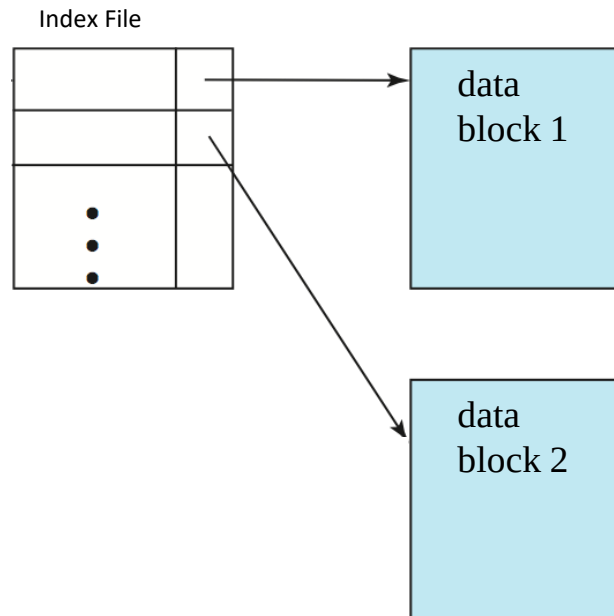
■ Sparse Index

- +: Less space and less maintenance overhead for INSERT and DELETE
- -: Generally slower than Dense Index for locating records.

■ Good tradeoff solution

- Sparse Index File with one index entry per block in the Data File:
 - 1st record in index points to the 1st record in the 1st block of the data file
 - 2nd record in index points to the 1st record in the 2nd block of the data file

- ...



Secondary Index Files

■ Secondary Index Files

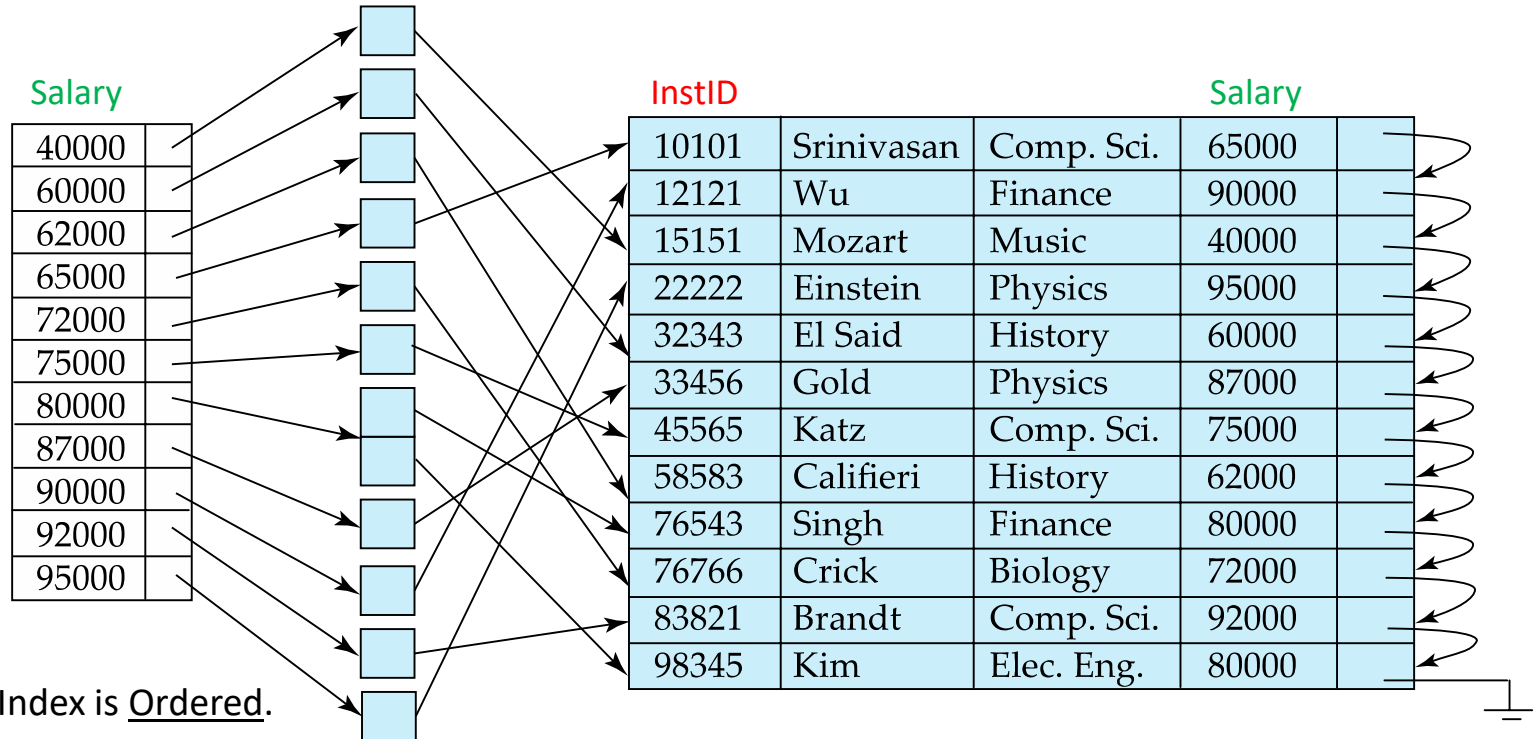
- Help finding all records whose values in a certain field satisfy some condition.
- The Instructor table is stored sequentially by InstID, finding all instructors in a particular department is easy with a Secondary Index on DeptName.
- A Secondary Index on Salary is helpful finding instructors with a specified salary or with a salary in a specified range of values.

■ Secondary Index Files and MySQL/MariaDB

- MySQL/MariaDB will automatically make a Primary Index for the Primary Key and Secondary Indexes for all Foreign Keys.

Secondary Index

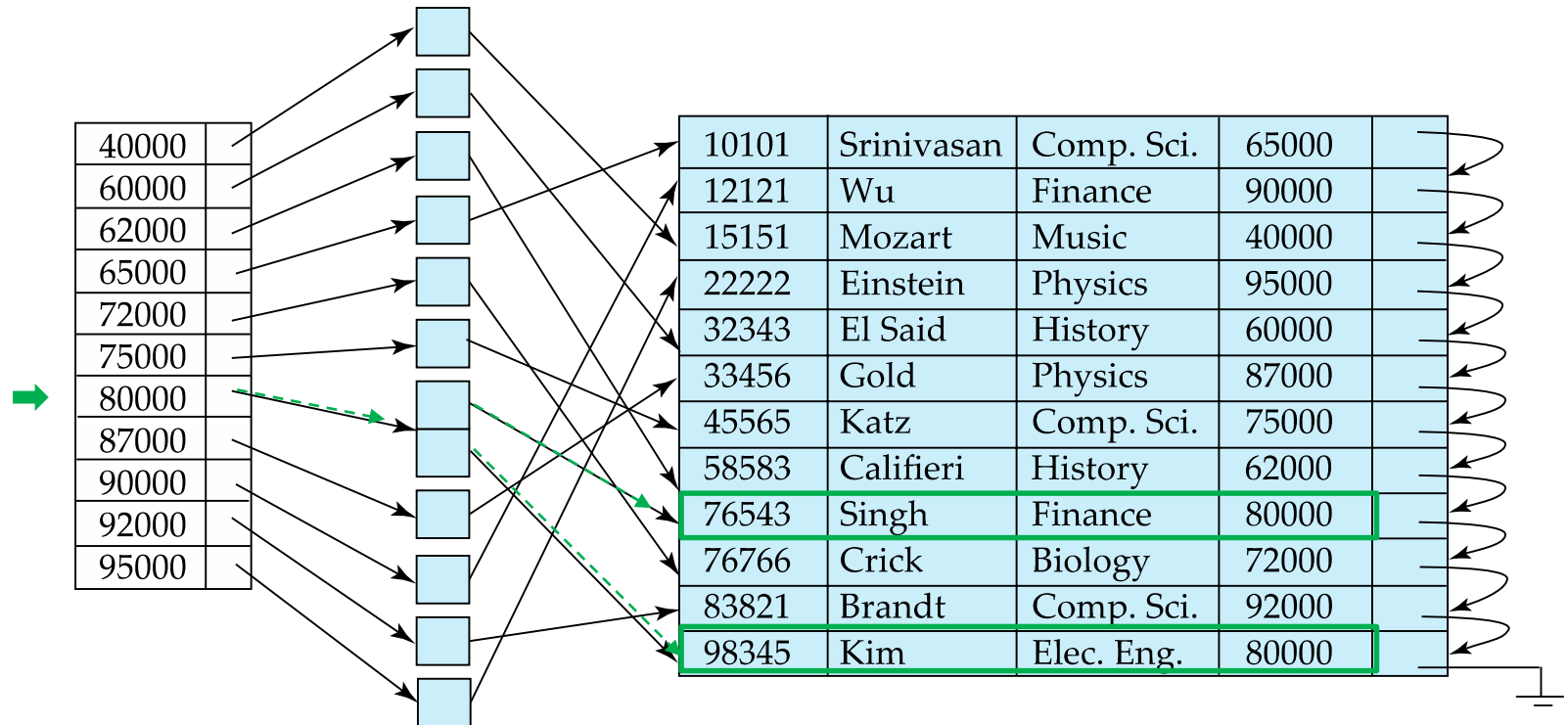
- Secondary Index on **non primary key**: Instructor.Salary (assume primary index on InstID)



- The Index is Ordered.
- The Index is Secondary (Index File and Data File are sorted on different Search Keys: **Salary** and **InstID**).
- Secondary Indexes are always Dense (All File Search Keys have an Index File Record).
- For each Search Key value K in the Index File, the associated pointer points to a bucket that contains pointers to all the data file records with that value for the Search Key. (If the index had been on a candidate key, buckets were not needed.)

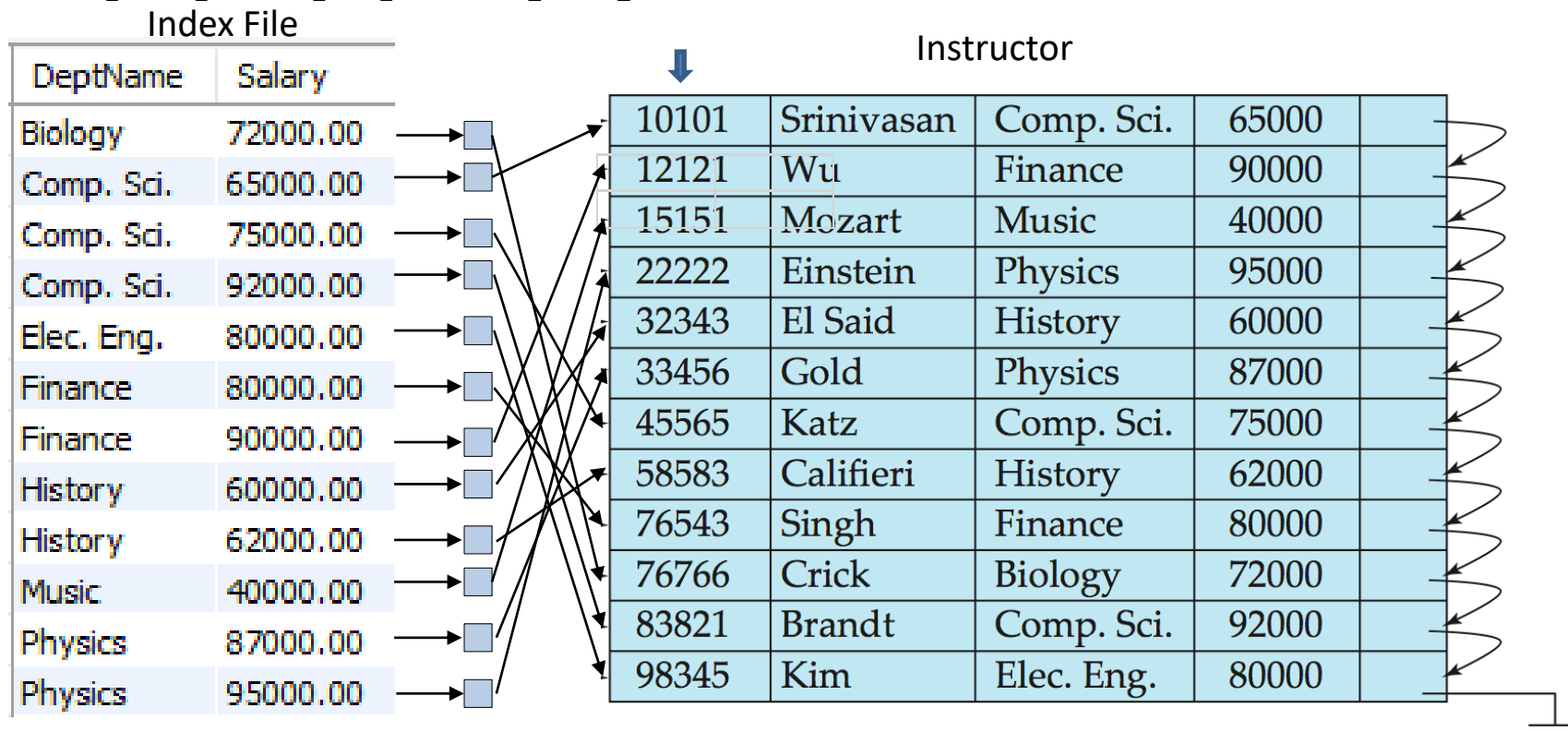
Querying Using a Secondary Index

- To locate a record with Search Key value K (e.g. Salary = 80000)
 1. Find the record with Search Key value $V = K$ in the Index File.
In practice: read the index into memory and use binary searching.
 2. Follow the associated pointer to the associated bucket and follow each of the bucket pointers to a record in the Data File and read the record (block containing it) into main memory.



Multi Column Index

- has **Composite search keys** containing more than one attribute
 - E.g. a secondary index on instructor(DeptName, Salary) or a primary index on Classroom(Building, Room).
- Index file is **lexicographically ordered**: $(a_1, a_2) < (b_1, b_2)$ if either
 - $a_1 < b_1$, or $a_1 = b_1$ and $a_2 < b_2$



Searching Using a Multi Column Index

- If the table has a multiple-column index on (A1, A2, ..., An), it provides indexed search capabilities on:
 - (A1), (A1, A2), ..., and (A1, A2, ..., An).
- Example: having a secondary index on (DeptName, Salary) for Instructor facilitates queries like:
 - **SELECT ... FROM Instructor WHERE DeptName = "Finance" AND Salary = 80000**
 - **SELECT ... FROM Instructor WHERE DeptName = "Finance"**
- but not a query like
 - **SELECT ... FROM Instructor WHERE Salary = 80000**

Primary and Secondary, Ordered Index Files

- Substantial time saving benefits in searching for records
 - Searching using Primary, Ordered Index File is efficient.
 - Searching using Secondary, Ordered Index File is less efficient:
 - Each record access may fetch a new Block from disk!
- However
 - When a Data File is modified the Index Files must also be updated.
 - Updating Index Files imposes an overhead on database modification.

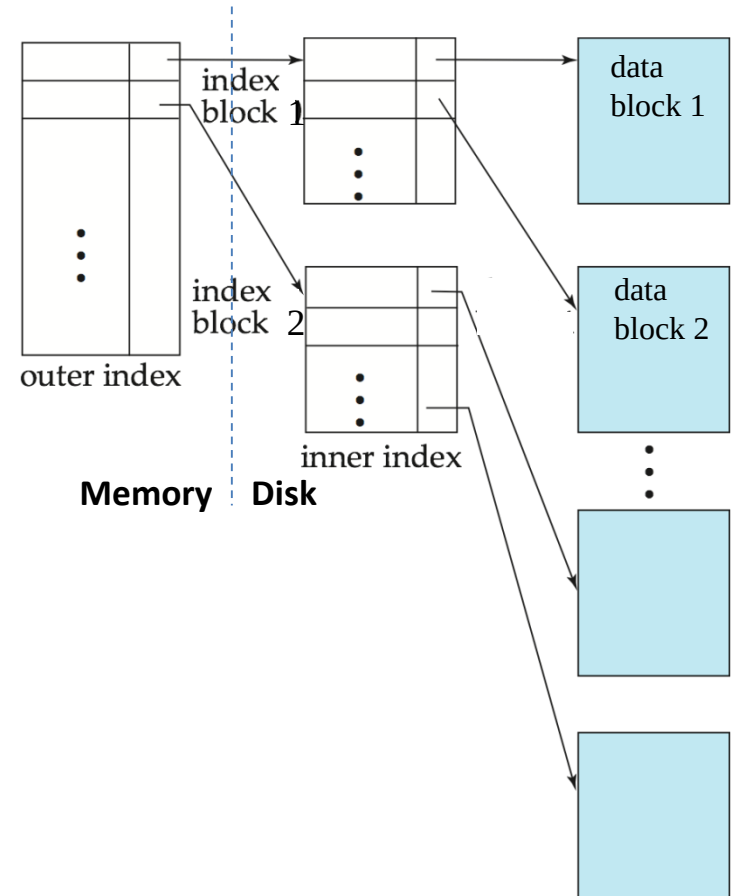
Multilevel Index

- When the Index File gets too big to fit in memory!

- Create a new index for the Index File!

- Solution: Build an Outer Index

- Inner index:
The original Index File
- Outer index:
A Sparse Index of the Index File
- If the outer Index File gets too large to fit in memory, yet another level of index can be created.
- However, indexes at all levels must be managed on record insertion or deletion.

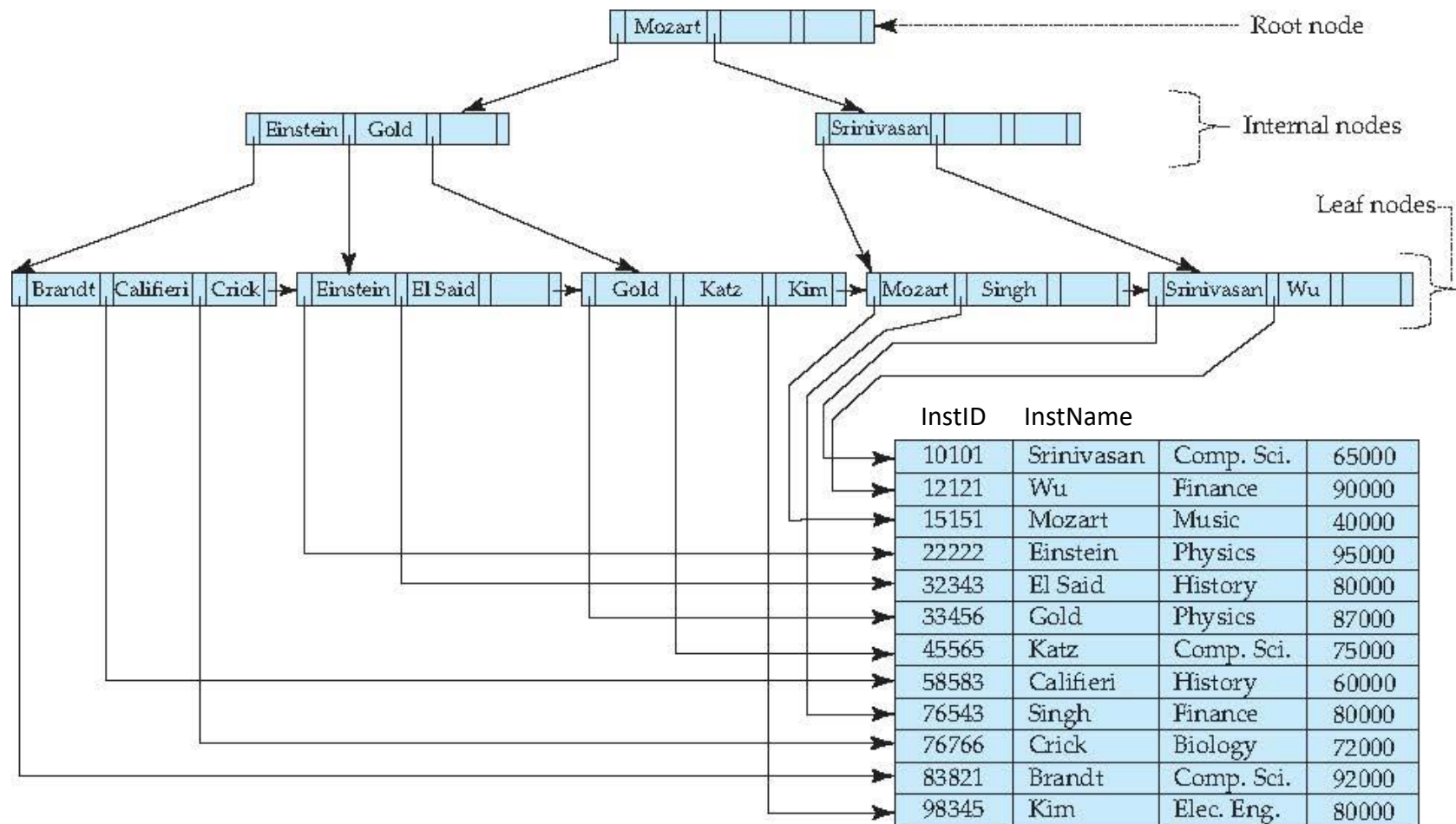


B⁺-Tree Index Files

- A *B+ Tree Index File* is a dynamic multi-level index structured as a *B+ tree*.
- *B+ Trees* are special cases of tree data structures.
- Disadvantage of Index-Sequential Files
 - Performance degrades as file grows (e.g. due to overflow blocks)
 - Periodic reorganization of the entire file is required
- Advantage of a B⁺-Tree Index File
 - Automatically reorganizes itself with small, local, changes after insertions and deletions.
 - Reorganization of the entire file is never required.
- Minor disadvantage of B⁺-Trees
 - Extra insertion and deletion overhead, space overhead
- Advantages of B⁺-Trees outweigh disadvantages
 - B⁺-Trees are used extensively in most database systems although they call them B-Trees!

Example of B⁺-Tree Index of Order 4 for Instructor with Search Key InstName

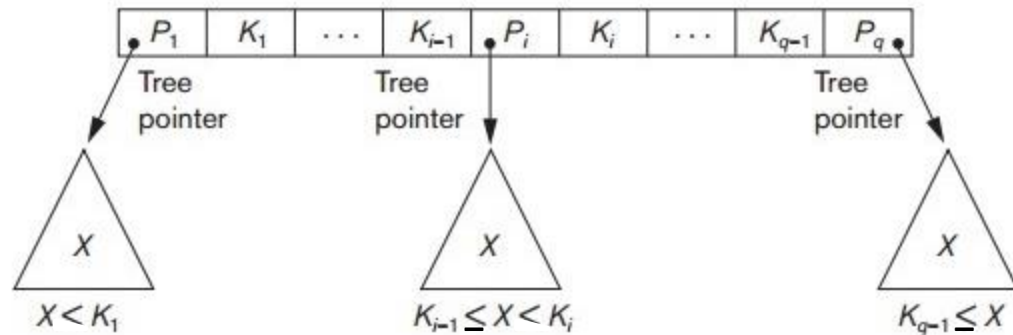
- Each Record has max 4 Pointers and max $4-1 = 3$ Search Keys in sorted order.
- Leaf nodes form a **Dense Secondary Index** (ordered by InstName) to the records in the data file.
- Non-leaf nodes at depth i form a **Sparse Index** to the nodes at depth $i + 1$.
- Read record for El Said?



B⁺-Tree of Order n

■ The B⁺-Tree constraints and properties

- Each node has one more pointer than search key values, except last leaf node.
- The tree must be **sorted**:
 - Keys must be in sorted order in each node: $K_1 < \dots < K_{q-1}$. (Assuming no duplicate search keys.)
 - The keys X in subtrees must satisfy:



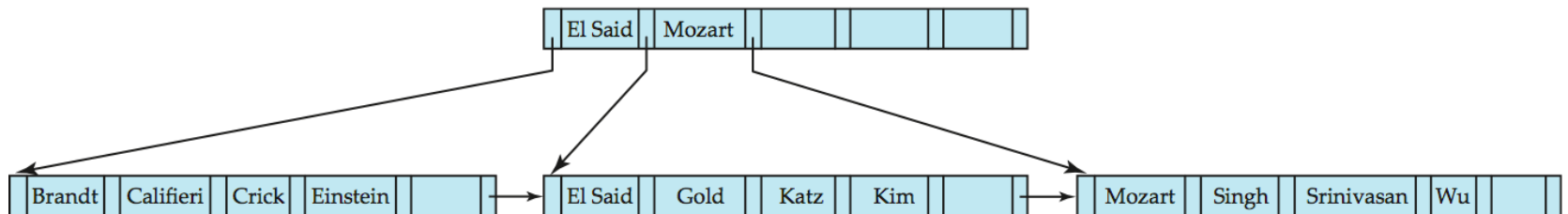
- The leaf nodes must be ordered, so $K_i < K_j$ for $K_i \rightarrow K_j$
- The tree must be **balanced** ("B" in the B⁺-Tree stands for Balanced), i.e. all paths from the root to a leaf are of the same length.
- **Filling of non-root nodes**:
 - Each **internal node** has min $\lceil n/2 \rceil$ and max n pointers to subtrees / (children) and min $\lceil n/2 \rceil - 1$ and max $n - 1$ search key values.
 - A **leaf node** has min $\lceil (n - 1)/2 \rceil$ and max $(n - 1)$ search key values and pointers to records in the datafile.
 - Above $\lceil x \rceil$ denotes the least integer i for which $x \leq i$.

The basic idea is to leave room in the nodes for new search keys.

Example of B⁺-Tree of Order 6

- B⁺-Tree with $n = 6$, that is max 6 pointers in each node.
 - Leaf nodes must have between 3 and 5 search key values (i.e. between $\lceil (n - 1)/2 \rceil$ and $n - 1$).
 - Non-leaf nodes other than root must have 3 to 6 pointers/children (i.e. between $\lceil n/2 \rceil$ and n), and must have 2 to 5 search key values.

B⁺-Tree for *instructor* file ($n = 6$)



Queries on B⁺-Trees

- Find record with search-key value V . (Assuming no duplicate search values.)

$C = \text{root}$

while C is not a leaf node

let i be least value s.t. $V \leq K_i$ (so $K_{i-1} < V$)

if no such exists, set $C = \text{last non-null pointer in } C$

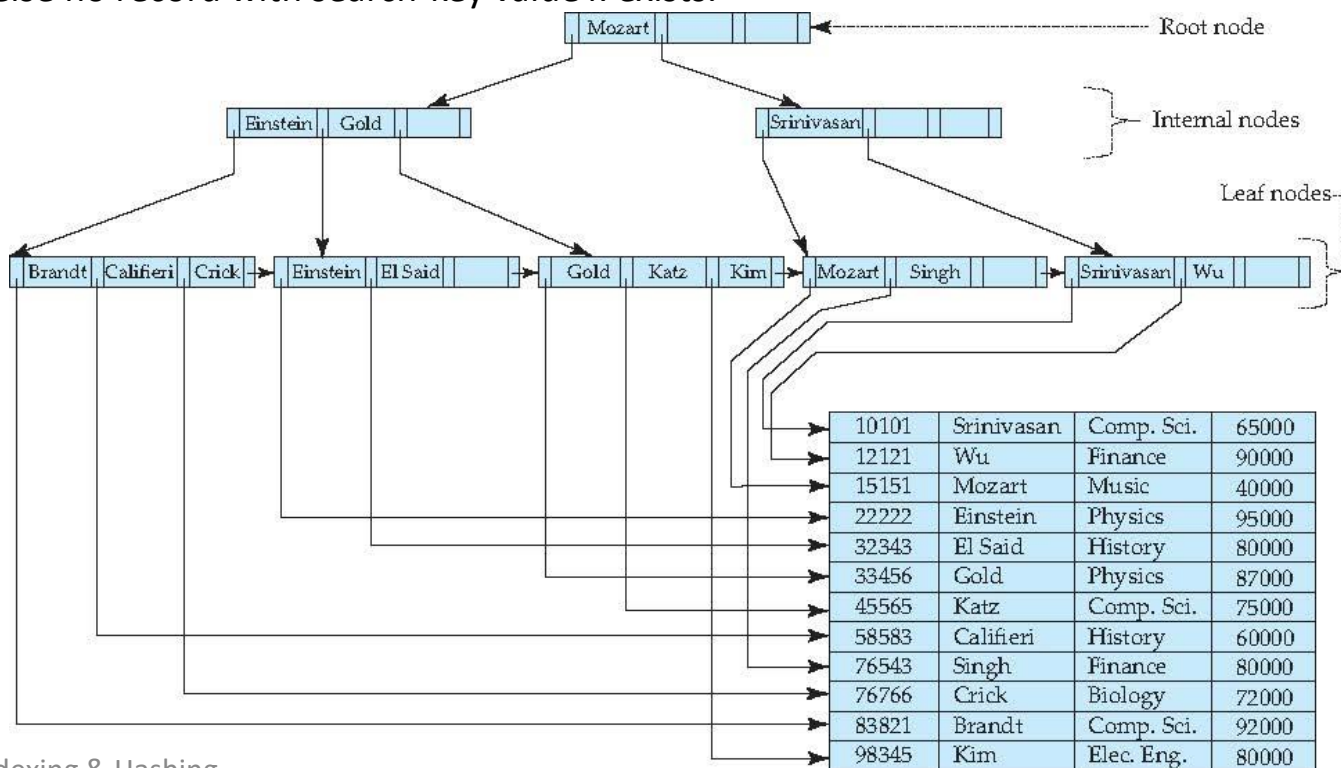
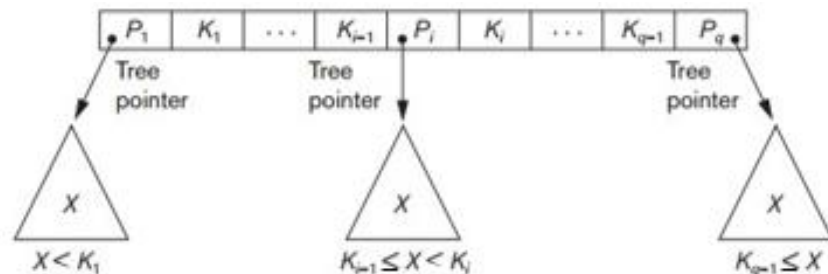
else if ($V = K_i$) set $C = P_{i+1}$

otherwise set $C = P_i$

let i be least value s.t. $K_i = V$

if there is such a value i , follow pointer P_i to the desired record.

else no record with search-key value k exists.



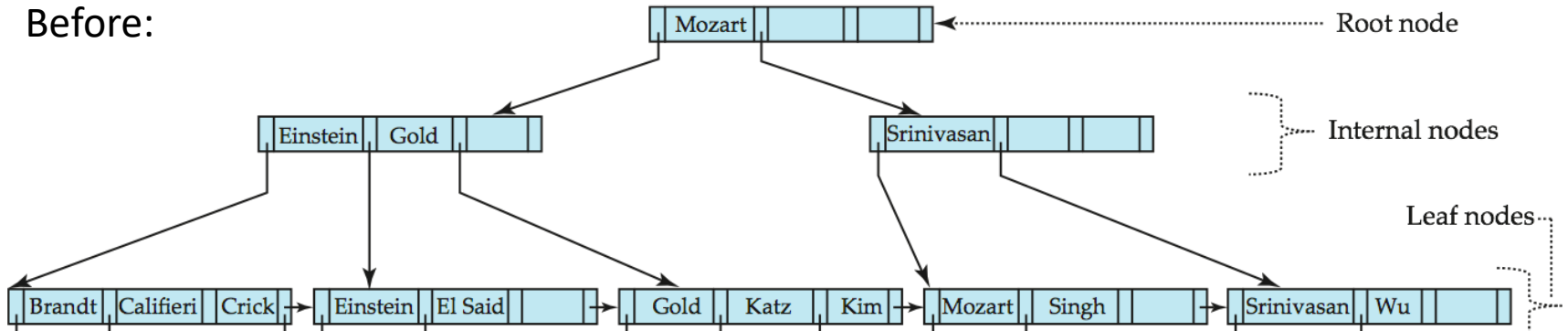
Queries on B⁺-Trees of order n

- If there are K search-key values in the file, the height of the tree is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
- A node is generally the same size B as a disk block.
- Example:
 - With $K = 1 \text{ million}$ search key values and $n = 100$ at most $\log_{50}(1,000,000) = 4$ nodes are accessed in a lookup.
 - Contrast this with a *balanced binary tree* with 1 million search key values — around $\log_2(1,000,000) \sim 20$ nodes are accessed in a lookup.
 - Above difference is significant since every node access may need a disk I/O, costing typically around 10-20 milliseconds.

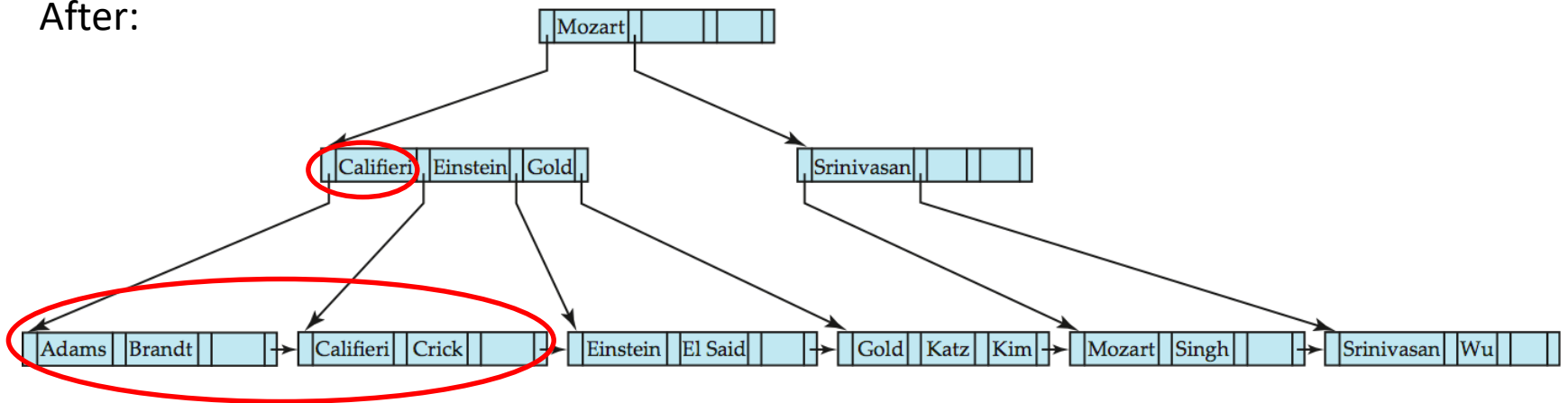
B⁺-Tree Insertion

B⁺-Tree before and after insertion of Adams!

Before:



After:



Observations about B⁺-Trees

■ Major Observations

- The leaf nodes form a **Dense Index**.
- The non-leaf nodes form a **Sparse Index** to the leaf nodes.
- The B⁺-Tree is effective for queries and modification.

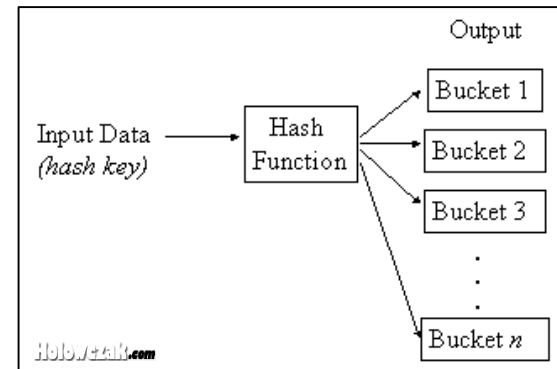
■ Detailed Observations

- Contains a relatively small number of levels.
- This means that searches are conducted efficiently.
- Insertions and deletions to the data file can be handled efficiently, with a minimum overhead of index manipulation.

Static Hashing

■ Principles

- A **bucket** is a unit of storage containing one or more records, a bucket is typically a disk block.
- In hashing the bucket of a record is found directly: it is calculated from its search key value using a hash function.
- A **hash function** is a function mapping each search key value (also called a **hash key**) to a bucket number.



- The hash function is used to locate records for access, insertion and deletion: **Record with search key k is placed in bucket with number $\text{hash}(k)$.**
- When hashing is used, the data file is said to be a **hash organized file**.
- Records with different search key values may be mapped to the same bucket; Thus the entire bucket has to be searched sequentially to locate a record.

Example of a Hash File Organization for Instructor Table

- Search key: DeptName (a text).
- Number of buckets: 8, numbered 0, 1, ..., 7.
- Hash function: $h(text) = (int(letter_1(text)) + \dots + int(letter_n(text))) \% 8$.
where *int* maps a letter to its alphabetic number.

$x \% 8$ will provide
buckets 0 to 7!

Example:

$h(\text{Music}) =$

$(13+21+19+9+3)\%8 =$

$65 \% 8 = 1$

Examples:

- $h(\text{Music}) = 1$
- $h(\text{History}) = 2$
- $h(\text{Physics}) = 3$
- $h(\text{Elec. Eng.}) = 3$

Bucket 0

Bucket 1

15151	Mozart	Music	40000

Bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

Bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

Bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

Bucket 5

76766	Crick	Biology	72000

Bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

Bucket 7

Hash Functions

■ Good and bad hash functions

- **Worst case of hashing:** all records are assigned to the same bucket.
- **Ideal case of hashing:** each bucket will have the same number of records assigned (record distribution to buckets is uniform). Hard to achieve as the distribution of search keys in the records may change over time.
- An **ideal hash function** h
 - is **uniform**, i.e. each bucket is assigned the same number of search key values from the set of all possible key values. Example: if there are 5 buckets and 20 search key values, the h should map 4 keys to each bucket.
 - is **random**, i.e. independent actual distribution of search key values in the file (so each bucket will have the same number of records assigned to it irrespective of the actual distribution of search key values in the file).

■ Typical hash functions

- Perform computation on the internal binary representation of the search key.
- For example, for a string search key, the binary representations of all the characters in the string could be added and the remainder of the sum modulo the number of buckets could be returned.

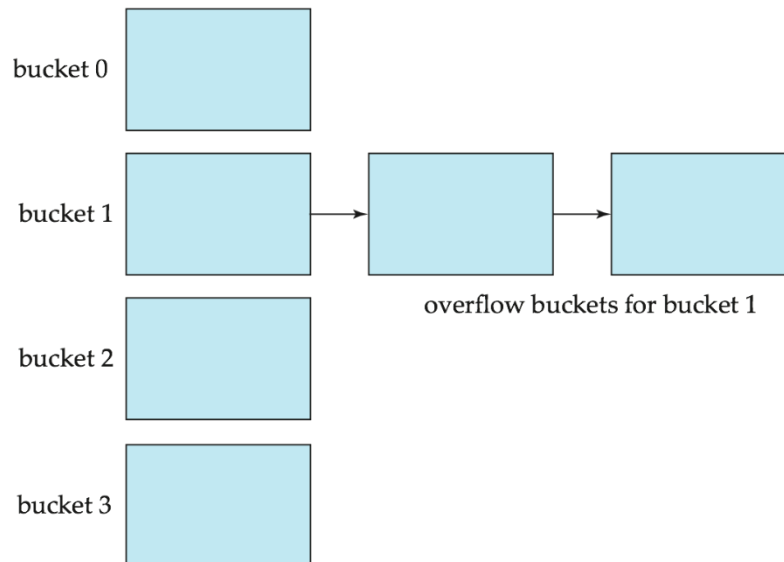
Handling of Bucket Overflows

■ Bucket overflow can occur

- Insufficient number of buckets.
- Skew in distribution of records due to
 - Multiple records have the same search key value.
 - Hash function produces non-uniform distribution of search key values.

■ Overflow buckets

- Although the probability of bucket overflow can be reduced, it cannot be eliminated; It is handled by using overflow buckets.



Hash Index Files

- Hash Index File
 - Hashing can also be used to make a stored **hash index file**, i.e. the index file is organized as a hash file: HashIndexFile(Search Key, Pointer)
 - Example: see next page.
 - Hash Index Files are always Secondary Index Files.

Example of Hash Index on *instructor*, on attribute *InstID*

bucket 0

76766	

bucket 1

45565	
76543	

bucket 2

22222	

bucket 3

10101	

bucket 4

bucket 5

15151	
33456	

bucket 6

83821	

bucket 7

12121	
32343	

Hash function $h(\text{InstID}) = \text{digitsum}(\text{InstID}) \% 8$
gives bucket in which index pointer for InstID is kept.
Buckets have size 2.

InstID

76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
33465	Gold	Physics	87000

Deficiencies of Static Hashing

■ Static hashing

- The hash function maps the search key values to a fixed set of bucket addresses. However, databases grow or shrink with time!
- If the number of buckets is too small, and the file grows, performance will degrade due to too much overflow.
- If the number of buckets is too large, space will be wasted.

■ One solution

- Periodic re-organization of the file with a new hash function.
- Expensive and disrupts normal operations.

■ Better solution

- Use **Dynamic Hashing** allowing the number of buckets to be modified dynamically. (This topic is not covered in these slides, but in the book. Is not part of the syllabus for this course.)

Hashing in MySQL/MariaDB

- Can declare a table to be stored in a **hash organized file**, like:

```
CREATE TABLE employees (  
    id INT NOT NULL,  
    fname VARCHAR(30),  
    lname VARCHAR(30),  
    job_code INT,  
    store_id INT )  
PARTITION BY HASH(store_id) PARTITIONS 4;
```

A record with store_id = 2005, will be placed in partition number $2005 \% 4 = 1$

- Can declare a **hash index** on a table, like:

```
CREATE INDEX NameIndex ON Instructor(InstName) USING HASH;
```

Choosing Indexes/Hashing

1. List *Search Key Candidates* (to potentially define indexes/ hashing on)
 - Primary keys (many DBMS automatically choose primary indexes on these)
 - Foreign keys (many DBMS automatically choose secondary indexes on these)
 - Attributes A_i that are often used for searching and sorting
(appear e.g. in join conditions, WHERE conditions, ORDER BY, ...)
2. Select which of the candidates should have an index/hash access
 - Use database statistics about frequencies of queries and data modifications, and table sizes.
 - Discuss with domain experts.
 - Never define indexes for small tables or for attributes that are often updated.
 - Trade off between
 - (1) shorter time for queries (SELECT)
 - (2) longer time for data modifications (INSERT, DELETE, UPDATE)
 - (3) space overhead for index data structures
3. Hashing versus indexing on A_i
 - If **SELECT** ..., A_i ... **FROM** table **WHERE** $A_i = c$ is common
hashing on A_i is better.
 - If *range queries* like **SELECT** ... **FROM** table **WHERE** $A_i \leq c_2$ **AND** $A_i \geq c_1$ are common, then *sequentially ordered indexes and B+ trees* are better.

It is all also a matter of your DBMS.



Summary

- How to avoid reading a data file sequentially to find a record during a query?
 1. Use **index files**. An index file is an additional file that contains information about where records are stored in a data file.
Examples of index file organizations: sequential, B⁺-trees, hash.
 2. Use **hashing**. Uses a hash function instead of an index file to locate a record in the data file.
- What to choose is a tradeoff between (1) access time and (2) insertion time, deletion time and space overhead.

Demo Exercises

Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Do the
Demo Exercises.



Ordered Indexes

Student Table for 11.1 and 11.2,

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58

11.1 Ordered, Primary and Dense Index

Make an Ordered, Primary and Dense Index for the Student table assuming a maximum of 4 records in each block.

(Instead of drawing the pointer, you can give the address it is pointing to, by a block# and the record# in that block.)

11.2 Ordered, Primary and Sparse Index

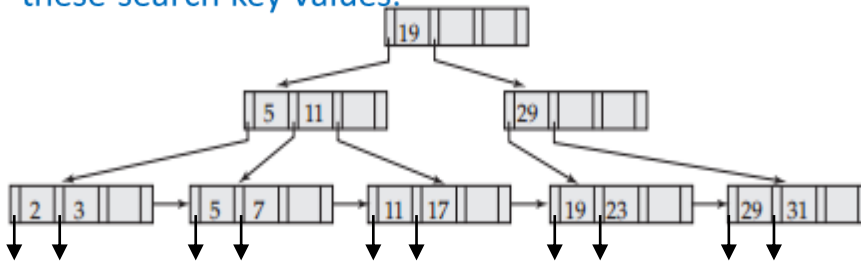
Make an Ordered, Primary and Sparse Index for the Student table assuming a maximum of 3 records in each Block.

(Instead of drawing the pointer, you can give the address it is pointing by a block# and the record# in that block.)

B⁺-Tree Indexes

11.3 B⁺-Tree

A file has the following set of search key values: (2, 3, 5, 7, 11, 17, 19, 23, 29, 31). Consider the following B⁺-Tree used as an index containing these search key values.



Show the steps involved in the two following queries:

- Find records with a search-key value of 11.
- Find records with a search-key value between 7 and 17, inclusive.

Hashing

11.4 Hash File Organization (Hashed Data File)

Suppose that hashing (a hash file organization) with search key A should be used for a data file of a relation R(A,B). There are 8 buckets numbered 0-7, each can hold max 3 records.

The hash function is: $h(x) = x \% 8$.

a) Show the contents of the 8 buckets after insertion of the following records of (A,B) pairs:

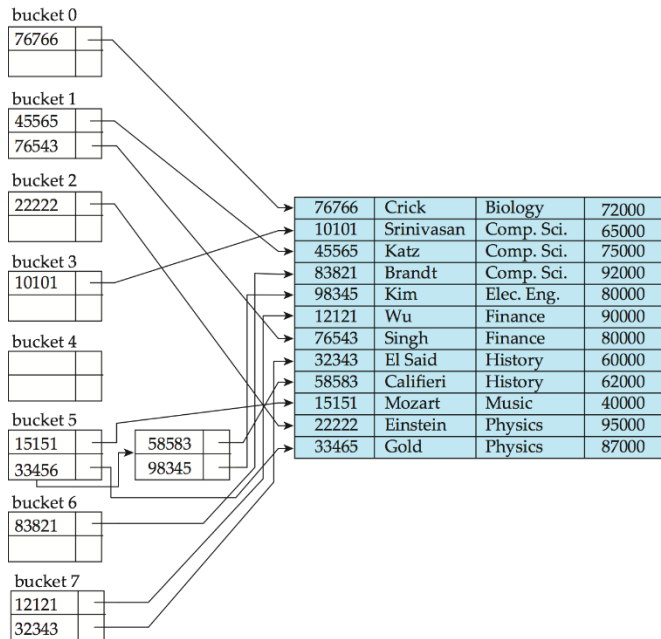
(2, X), (3, H), (5, P), (7, P), (11, Y), (17, C),

(19, D), (23, F), (29, X), (31, J)

b) After that, where should record (15, H) be added

Hashing

11.5 Hash Index (Hashed Index File)



Consider the hash index on InstID for Instructor, shown above. It uses hash function where e.g. $h(58583) = 5$.

Show the steps involved in the query “find the record with InstID = 58583”:

`SELECT * FROM Instructor WHERE InstID = 58583;`

Solutions to Demo Exercises



Ordered Indexes

Student Table for 11.1 and 11.2,

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58

11.1 Ordered, Primary and Dense Index

Make an Ordered, Primary and Dense Index for the Student table assuming a maximum of 4 records in each block.

(Instead of drawing the pointer, you can give the address it is pointing to, by a block# and the record# in that block.)

11.2 Ordered, Primary and Sparse Index

Make an Ordered, Primary and Sparse Index for the Student table assuming a maximum of 3 records in each Block.

(Instead of drawing the pointer, you can give the address it is pointing by a block# and the record# in that block.)

11.1 StudentIndex(Search Key, Block#, Record#)

Ordered:

Index is a sorted list

Primary:

Index and file are sorted on the same Search Key

Dense:

All file records have an index record.

Search Key	Block#	Record#
00128	1	1
12345	1	2
19991	1	3
23121	1	4
44553	2	1
45678	2	2
54321	2	3
55739	2	4
70557	3	1
76543	3	2

11.2 StudentIndex(Search Key, Block#, Record#)

Ordered:

Index is a sorted list

Primary:

Index and file are sorted on the same Search Key

Sparse:

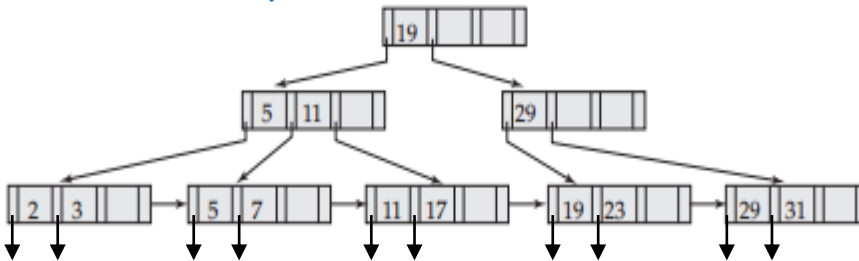
Only first record from each block record.

Search Key	Block#	Record#
00128	1	1
23121	2	1
54321	3	1
76543	4	1

B⁺-Tree Indexes

11.3 B⁺-Tree

A file has the following set of search key values: (2, 3, 5, 7, 11, 17, 19, 23, 29, 31). Consider the following B⁺-Tree used as an index containing these search key values.



Show the steps involved in the two following queries:

- Find records with a search-key value of 11.
- Find records with a search-key value between 7 and 17, inclusive.

- Find records with a value of 11
 - Search the first level index; follow the first pointer.
 - Search next level; follow the third pointer.
 - Search leaf node; follow first pointer to records with key value 11.

- Find records with value between 7 and 17, inclusive.

- Search top index; follow first pointer.
- Search next level; follow second pointer.
- Search third level; follow second pointer to records with key value 7, and after accessing them, return to leaf node.
- Follow fourth pointer to next leaf block in the chain.
- Follow first pointer to records with key value 11, then return.
- Follow second pointer to records with key value 17.

In this way 3 records are found.

Hashing

11.4 Hash File Organization (Hashed Data File)

Suppose that hashing (a hash file organization) with search key A should be used for a data file of a relation R(A,B). There are 8 buckets numbered 0-7, each can hold max 3 records.

The hash function is: $h(x) = x \% 8$.

a) Show the contents of the 8 buckets after insertion of the following records of (A,B) pairs:

(2, X), (3, H), (5, P), (7, P), (11, Y), (17, C),
(19, D), (23, F), (29, X), (31, J)

b) After that, where should record (15, H) be added?

11.4 Hash File Organization (Hashed Data File)

x	2	3	5	7	11	17	19	23	29	31
h(x)	2	3	5	7	3	1	3	7	5	7

Bucket 1 contains (17, C)

Bucket 2 contains (2, X)

Bucket 3 contains (3, H), (11, Y), (19, D)

Bucket 5 contains (5, P), (29, X)

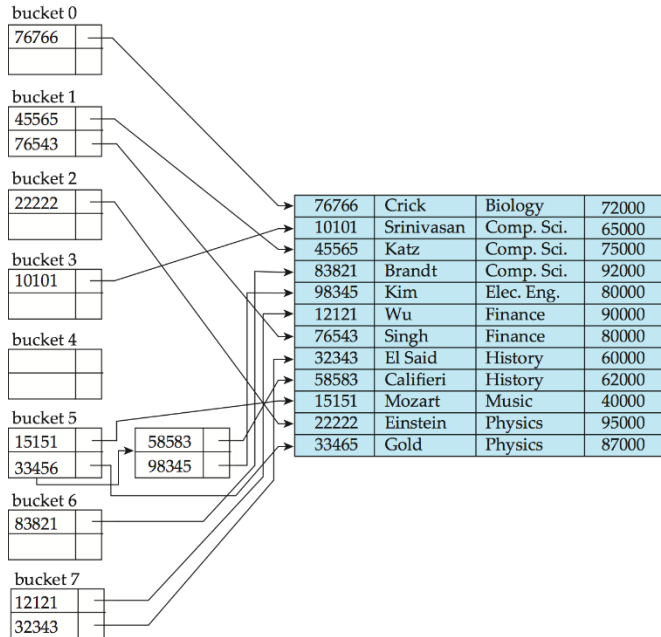
Bucket 7 contains (7, P), (23, F), (31, J)

Buckets 0, 4, and 6 are empty.

b) (15, H) should be added to bucket 7, but there is no space as it already has 3 records. The record must be added in an **overflow bucket** for bucket 7.

Hashing

11.5 Hash Index (Hashed Index File)



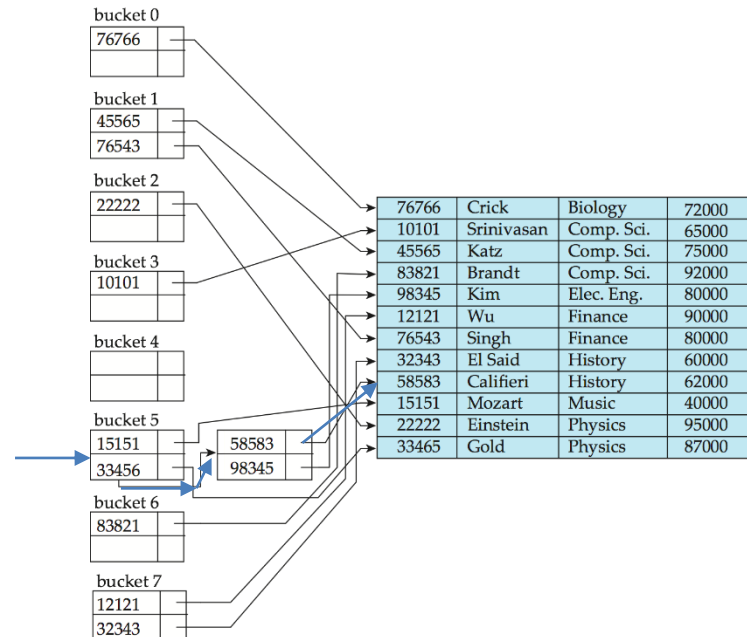
Consider the hash index on InstID for Instructor, shown above. It uses hash function where e.g. $h(58583) = 5$.

Show the steps involved in the query “find the record with InstID = 58583”:

`SELECT * FROM Instructor WHERE InstID = 58583;`

11.5 Hash Index (Hashed Index File)

$h(58583) = 5$, so the index pointer for 58583 is searched for in bucket 5. It is not in the main part of bucket 5, so the pointer to the overflow bucket of bucket 5 is followed and the searching is continued in that bucket. The record is found as the first. The pointer of this record is followed to the record in the data file having InstID = 58583 (and then all its attributes are selected).



Exercises

Please answer all exercises
to demonstrate your
Database Skills.

Pencil and Paper Exercises
Solutions are available at 11:45



Indexes

Student Table

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58

11.6 Ordered, Secondary and Dense Index

Make a Sequentially Ordered, Secondary and Dense Index for the Student table with Search Key DeptName.

Assume that the data file has 4 records in each block.

11.7 B⁺-Tree Index

A file has the following set of search key values: (2, 3, 5, 7, 11, 17, 19, 23, 29, 31). Consider the following B⁺-Tree used as an index containing these search key values.



- What is the order n of the B⁺ Tree?
- What is the allowed number of search keys in a non-root node in a B⁺ tree of that order?
- Show the steps involved in the two following queries:
 - Find records with a search-key value of 11.
 - Find records with a search-key value between 7 and 17, inclusive.

Hashing

11.8 Hashed Data File

Make a hash file organization for the Student table with Search Key StudID, assuming that records are inserted in the order shown in the table. There should be 4 buckets, which each can hold max 5 records.

What would the result be, if there could only be 4 records in a bucket?

11.9 Hash Index (Hashed Index File)

Make a hash index for the Student table with Search Key StudID, assuming that records are inserted in the order shown in the table. There should be 4 buckets, which each can hold max 5 records.